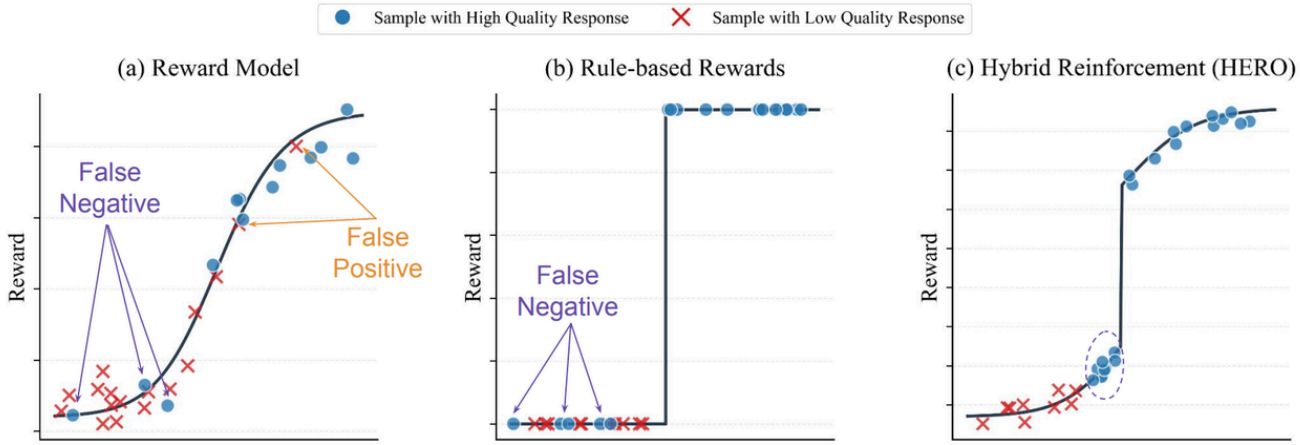
Hybrid Reinforcement: When Reward Is Sparse, It's Better to Be Dense

> #RewardModel #RL



- What problems does this framework solve?
  - Traditionally, there are two kinds of reward models: **sparse reward based on verifier** (binary $\in \{0, 1\}$) and **dense reward model** (e.g., 10-pts score).
  - The former model is reliable, but can also cause some fatal problems: if all trajectories in a group get the same score, if the GRPO is used, then the advantage will be 0 (lack of training signal). Also, it cannot distinguish the degree of correctness (correct, but how "correct" it is?).
  - The latter model is more agile, but it tends to be misled by bias, and using such models in RL will make policy models learn to hack the reward.
  - Hybrid reinforcement is to provide a reward model that combines the advantages of these two types together.
- What does this framework do?
  - Instead of only using either sparse reward or dense reward, we leverage both, but use the sparse reward to **constraint** the dense reward.
  - Let r_{rule} be the sparse reward and r_{RM} be the dense reward, we have

$$\hat{r}(x, y) = \begin{cases} -\alpha + 2\alpha \cdot \frac{r_{RM} - \min r_{RM}}{\max r_{RM} - \min r_{RM} + \epsilon}, & r_{rule} = 0 \\ (1-\beta) + 2\beta \cdot \frac{r_{RM} - \min r_{RM}}{\max r_{RM} - \min r_{RM} + \epsilon}, & r_{rule} = 1 \end{cases}$$

    - $\alpha, \beta \in (0, 1]$ are hyperparameters to limit the bounds of the rewards. They are normally relatively closer to 0 to make r_{RM} is an auxiliary signal to the 0-1 based reward (or, to make the 0-1 *softer*).
    - If the answer is **incorrect** (verifier = 0), then we normalize the dense reward and use it to calculate $\hat{r} \in [-\alpha, \alpha]$.
    - If the answer is **correct** (verifier = 1), then we normalize the dense reward and $\hat{r} \in [1-\beta, 1+\beta]$.
  - Plus, traditionally, it is regarded that all prompts are equally weighted in RL. However, this work proposes that some of the prompts are "more valuable" than others, i.e., they are neither too

hard or too easy. Mathematically, the **dense rewards** of such type of prompts have a high variance.

- Thus, we can use the variance to produce a dynamic weight for the reward:

  $w_{difficulty}(\sigma_u) = w_{min} + (w_{max} - w_{min}) \cdot \frac{1}{1+\exp(-k(\sigma_u - \bar{\sigma}))}$

  - If the prompt/question is valuable, aka $\sigma_u = \mathrm{std}(r_{RM})$ is large, then the weight will be high $\approx w_{max}$.
  - Otherwise, it will be $\approx w_{min}$.

- The final reward will be:

  $r_{final}(x, y) = w_{difficulty}(\sigma_u) \cdot \hat{r}(x, y)$

- Experiment Results
  - Comprehensive Outperformance of Baseline: HERO consistently outperforms pure RM training and pure rule training across fully verifiable tasks, hard-to-verify tasks, and mixed tasks.
  - Negative Feedback Domain Proves More Critical: Ablation experiments reveal that providing dense negative feedback intervals (i.e., the $[-\alpha, \alpha]$ range of incorrect answers in the aforementioned formula) contributes more significantly to stable training and accuracy improvement than providing positive feedback intervals. This approach better penalizes different types of reasoning errors.
  - Independent of Massive RM: Scaling RM from 7B to 72B yields no significant gains and even shows slight deterioration on hard-to-verify tasks. This demonstrates that HERO's performance boost primarily stems from its sophisticated fusion architecture design (hierarchical normalization and reweighting), rather than relying on the brute-force stacking of reward model parameters.

- Work Cited
  - Paper
  - Chat History
  - NotebookLM